

Solving 0-1 Knapsack Problems Using Sine-Cosine Algorithm

Khaled Mahfouz

*College of Engineering and Information Technology
Ajman University
Ajman, UAE
k.mahfouz@ajman.ac.ae*

Mohammed Azmi Al-Betar

*College of Engineering and Information Technology
Ajman University
Ajman, United Arab Emirates
m.albetar@ajman.ac.ae
Department of Information Technology
Al-Huson University College
Al-Balqa Applied University
Irbid, Jordan
mohbetar@bau.edu.jo*

Sharaz Ali

*College of Engineering and Information Technology
Ajman University
Ajman, UAE
sharaz.ali@ajman.ac.ae*

Mohammed A. Awadallah

*Department of Computer Science
Al-Aqsa University
P.O. Box 4051, Gaza, Palestine
Artificial Intelligence Research Center (AIRC)
Ajman University
Ajman, United Arab Emirates
ma.awadallah@alqsa.edu.ps*

Abstract—The task of optimization is no easy task and from a computational point of view, it often involves scanning a large search space to find the best solution that adheres to all the constraints and desired specifications. Designing a customized algorithm to solve several optimization problems is also a challenging task, therefore scientists and engineers utilize meta-heuristic algorithms that can provide an optimal solution within a reasonable time. This optimal solution may or may not be the best solution in the search space, but it is usually good enough to satisfy the requirements without spending a lot of computational resources or time. The 0-1 knapsack problem is an constraint-based optimization problem in which a number of items have to be packed into a container by maximizing the value of the items in the container while also adhering to the weight limit of the container. In this paper, sine-cosine algorithm (SCA) is adopted to solve 0-1 knapsack problems. The proposed algorithm is called binary sine-cosine algorithm (BSCA). Due to the binary nature of 0-1 knapsack problem, the SCA is manipulated using a mapping function. The performance of the proposed BSCA is evaluated using 15 well-known datasets. Furthermore, the performance of the proposed BSCA is compared with other comparative algorithms (i.e., GA, PSO, and BFPA) from the literature using the same datasets. It can be observed from the results that the performance of the proposed BSCA is similar to other algorithms by obtaining the optimal results on 10 datasets. While the results of the proposed BSCA are convergent with others for the remaining five datasets.

Index Terms—Optimization, Constrained optimization, Sine-Cosine Algorithm, Meta-heuristic, Population-based algorithm, Knapsack problem

I. INTRODUCTION

The concept of optimization is an integral part of our everyday lives. Whether it's businesses trying to maximize profits

with minimal investment or it's engineers trying to minimize the drag force on an aircraft wing while maximizing the lift force, there are many optimization problems to solve in the real life. In addition, optimization problems can also contain constraints that need to be satisfied in addition to maximizing or minimizing some design parameters [1]. Therefore the task of optimization is no easy task and from a computational point of view, it often involves scanning a large search space to find the best solution that adheres to all the constraints and desired specifications. Designing a customized algorithm to solve each optimization problem is also a challenging task, therefore scientists and engineers utilize meta-heuristic algorithms that can provide an optimal solution within a reasonable amount of time. This optimal solution may or may not be the best solution in the search space, but it is usually good enough to satisfy the requirements without spending a lot of computational resources or time.

Meta-heuristic algorithms come in two primary flavors: evolutionary and swarm intelligence. Evolutionary algorithms are the oldest meta-heuristic algorithms and they are heavily influenced by concept of evolution in nature and they try to mimic those ideas such as mutation and gene crossover from parent individuals. The most famous evolutionary algorithm is Genetic Algorithm (GA) [2]. Swarm intelligence techniques mimic the behaviour of animals or individuals in colony on collecting their foods. There are many swarm intelligence techniques particle swarm optimization [3], horse herd optimization algorithm [4], group search optimizer [5], salp swarm optimization [6], and others reported in [7], [8].

The Sine Cosine Algorithm (SCA) is a recent meta-heuristic

algorithm designed by Mirjalili [9]. This algorithm like other meta-heuristic algorithms start with a set of random solutions. Iteratively, these solutions are guided towards the best solution in the population using sine and cosine operators. This algorithm is successfully applied to solve many optimization problems summarized in [10].

The 0-1 knapsack problem is a constraint-based optimization problem in which, a set of items, with given weights and profits, need to be placed into a container with a maximum capacity. In that case, the problem is to choose a subset of the items of maximum total profit that can fit in the container. The 0-1 in the knapsack problem means that an object can either be placed in the container (indicated by 1) or not (indicated by 0). Partial objects cannot be placed in the container. There are many metaheuristic algorithms utilized to solve 0-1 knapsack problem like grey wolf optimization [11], whale optimization algorithm [12], chemical reaction optimization [13], monarch butterfly optimization [14], etc.

In this paper, the SCA is utilized to solve 0-1 knapsack problem. Due to the binary nature of the problem, the SCA is modified to convert from continuous to binary context. The proposed algorithm is called binary sine-cosine algorithm (BSCA). For evaluation purposes, in total fifteen 0-1 knapsack dataset instances were extracted from [15]–[17]. These instances vary in sizes and complexity. The experimental results show the efficiency of the proposed BSCA against the other algorithms especially the small datasets.

The organization of this paper is as follows: Section II contains a review of existing literature related to evolutionary algorithms and their application on the 0-1 knapsack problem. Section III provides a description of the 0-1 knapsack problem and how the problem is modeled mathematically. Section IV gives a more detailed analysis of how the proposed BSCA is adapted to solve the 0-1 knapsack problem. Section V shows the results of the proposed BSCA in comparison with other algorithms. Finally, Section VI contains the conclusion and future directions.

II. LITERATURE REVIEW

This section discusses various metaheuristic algorithms introduced by researchers to solve 0-1 knapsack problem. Singh, R. P. [18] proposed the use of a rather simple GA to solve one 0-1 knapsack instance comprised of four items. The algorithm however was not extensively analyzed in terms of performance, neither with respect to other metaheuristic techniques nor in terms of its ability to solve larger 0-1 knapsack instances. However, Lim [19] proposed the use of monogamous pairs genetic algorithm (MopGA) to solve 0-1 knapsack problems that introduces operations derived from social monogamy such as pair bonding and infidelity. The author used a dataset of low-dimensional 0-1 knapsack instances similar to the ones used in this paper, but different high-dimensional ones but proved to perform better than state-of-art techniques specially when optimizing high-dimensional 0-1 knapsack instances. Unlike Li, Z., and Li, N. [20] who proposed their novel multi-mutation binary particle swarm

optimization algorithm (MMBPSO) that uses a probabilistic transfer function to binarize continuous search space, the previously mentioned GA-based techniques may not have required such a phase since operations are to happen on binary vectors already. MMBPSO used a single mutation operator that adjusts a particle in local bit with a random probability and a full mutation operator that performs a macroscopic mutation to all bits of a particle with a random probability to optimize four benchmark high-dimensional 0-1 knapsack instances, and was capable of escaping local optima and obtaining better results compared to other well-know variants of BPSO.

In [21], the authors proposed a binary version of dragonfly algorithm (BDA) to solve 0-1 knapsack problems inspired by the novel swarm intelligence optimization algorithm named dragonfly algorithm (DA) that utilizes the swarming nature of dragonflies. The algorithm used a probabilistic transfer function similar to the one used in BPSO to optimize 0-1 knapsack instances. Whereas, Abdel-Basset, M., et. al, [22] introduced a binary version of the flower pollination algorithm (FPA) for solving 0-1 knapsack instances by using a sigmoid function to map real-valued solutions to binary ones. Both algorithms used the same set of low-dimensional instances as the ones optimized in this paper, but dissimilar high-dimensional instances.

Needless to say, numerous authors proposed algorithms that perform outstandingly for both low and high dimensional 0-1 knapsack instances such as [23] who proposed a modified scheme of the binary artificial bee colony algorithm that adds a differential evolution phase to the existing binary artificial bee colony algorithm. Similarly, [24] proposed a binary version of a recently developed optimization algorithm named Marine Predators Algorithm (MPA) that investigates a range of transfer functions to help map a continuous search space to binary. They clearly mention that the use of different v-shaped and s-shaped transfer functions may be essential when solving different knapsack instances depending on the complexity of the dataset.

According to the experimental results, the proposed BSCA proved to perform as good as any of the previously mentioned approaches when optimizing many of the well-cited low-dimensional 0-1 knapsack instances in terms of acquiring the instance's optimum value and have performed better in terms of run-time to optimum for some of those instances compared to BFPA and BMPA. However, BSCA seemed to perform weaker as the dimension of the instance increases above 30 items.

III. PROBLEM MATHEMATICAL MODELING

Mathematically, we can define a vector x that contains binary values of either 0 or 1 which indicates if the object is included in the container or not:

$$x = (x_1, x_2, \dots, x_n), \quad x_i \in \{0, 1\} \quad (1)$$

In addition, we can define another vector w that contains all the weights of the items:

$$w = (w_1, w_2, \dots, w_n) \quad (2)$$

Moreover, we can define another vector p that contains all the profits or value of the items:

$$p = (p_1, p_2, \dots, p_n) \quad (3)$$

Upon defining the above three vectors, the fitness function can be formulated as a piecewise function, that incorporates the idea of using a fitness penalty in case the total weight of the items exceeds the capacity (i.e., c) of the container [25]:

$$f(\mathbf{x}) = \begin{cases} \sum_{i=1}^n p_i x_i, & \sum_{i=1}^n w_i x_i \leq c \\ c - \sum_{i=1}^n w_i x_i, & \sum_{i=1}^n w_i x_i > c \end{cases} \quad (4)$$

First, the combined weights of the selected objects is calculated. If the combined weights of the objects is less than the capacity of the container then the fitness function is defined as the combined profit values of the items. Else, a fitness penalty is applied, by calculating the difference between the capacity of the container and the combined weights of the items, i.e., the excess weight compared to the container capacity. The higher the excess weight is, the larger the penalty will be and the less likely that the particular solution is selected in the next iteration of search process.

IV. PROPOSED METHOD

The detailed description of utilizing the binary sine-cosine algorithm for solving 0-1 knapsack problem. Algorithm 1 illustrates the procedural steps of the proposed BSCA on solving 0-1 knapsack problem. Different phases of the proposed BSCA implementation will be discussed separately under sub-sections to help distinguish them.

Algorithm 1: The procedural steps of the proposed BSCA

```

1 Initialize the parameters of BSCA and 0-1 knapsack
  problem
2 Population Initialization
3 while  $t < t\_Max$  do
4   Improvement process
5   Solution mapping
6   Evaluate the quality of each solution
7   Position Updates
8 end
9 return the best solution
```

A. Step 1: Initialize the parameters of BSCA and knapsack problem

There are two control parameters in the proposed BSCA: the maximum number of iterations (i.e., t_Max), and the size of the population (i.e., PZ). Furthermore, the parameters of 0-1 knapsack problem are extracted from the datasets, where these parameters are mentioned in Section III.

B. Step 2: Population Initialization

The population is a matrix of size $PZ \times n$, where the PZ is the population size, and n is the number of items in the container. The population is formulated as follows:

$$\mathbf{X} = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,n} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,n} \\ \vdots & \vdots & \cdots & \vdots \\ x_{PZ,1} & x_{PZ,2} & \cdots & x_{PZ,n} \end{bmatrix} \quad (5)$$

A random stream of bits equal to the number of items in the 0-1 knapsack instance will be initialized for every candidate solution in the population. When optimizing any of the 0-1 knapsack instances, initialized or updated binary candidate solution will simply undergo a dot product with the weight and profit vector of the instance to evaluate its fitness whilst maintaining the weight constraint of the knapsack problem. If the weight constraint is violated, a penalty is applied to the fitness of the candidate solution according to the previously mentioned fitness function in the Problem Mathematical Modelling section of this paper (see Eq. (4)). Later on, the solutions in the population are sorted in ascending order, where $f(\mathbf{x}_1) \leq f(\mathbf{x}_2) \leq \dots \leq f(\mathbf{x}_{PZ})$.

C. Step 3: Improvement process

In this phase, each solution $\mathbf{x}_i$ in the population is modified based on sine and cosine operators as shown in Eq. (6) and Eq. (7).

$$\mathbf{x}_i(t+1) = \mathbf{x}_i(t) + r_1 \times \sin(r_2) \times |r_3 \mathbf{x}^{best}(t) - \mathbf{x}_i(t)| \quad (6)$$

$$\mathbf{x}_i(t+1) = \mathbf{x}_i(t) + r_1 \times \cos(r_2) \times |r_3 \mathbf{x}^{best}(t) - \mathbf{x}_i(t)| \quad (7)$$

Where $\mathbf{x}_i(t)$ is the current solution in i -th position at t -th iteration, r_1 , r_2 , and r_3 are random numbers between 0 and 1. $\mathbf{x}^{best}$ is the best solution in the population. These two equations are combined to be used as follows:

$$\mathbf{x}_i(t+1) = \begin{cases} \mathbf{x}_i(t) + r_1 \times \sin(r_2) \times |r_3 \mathbf{x}^{best}(t) - \mathbf{x}_i(t)|, & r_4 < 0.5 \\ \mathbf{x}_i(t) + r_1 \times \cos(r_2) \times |r_3 \mathbf{x}^{best}(t) - \mathbf{x}_i(t)|, & r_4 \geq 0.5 \end{cases} \quad (8)$$

where r_4 is a random between 0 and 1. The current solution $\mathbf{x}_i$ is either updated based on sine operator or cosine operator based on the value of r_4 .

D. Step 4: Solution mapping

It is very important to remember that SCA does not work with binary candidate solutions rather than floating-type decimal variable candidates. This stream of bits will be mapped to its equivalent representation in floating-type decimal using the mapping function described in Eq. (9) below. This mapping is essential for SCA to work since SCA functions on a continuous search-space by nature, i.e. position updates of candidate solutions happen in a continuous search-space.

$$\Gamma(a_1, \dots, a_L) = x + \frac{y-x}{2^L-1} \cdot \left(\sum_{j=0}^{L-1} a_{L-j} \cdot 2^j \right) \in [x, y] \quad (9)$$

E. Step 5: Position Updates

After the improvement process is achieved, the quality of the solution is calculated using Eq. (4). Later on, if the quality of the current solution $\mathbf{x}_i(t+1)$ is better than the old one $\mathbf{x}_i(t)$, then the new one replaced the old one.

F. Step 6: Stop condition

Step 3 to Step 5 are repeated until the t_{Max} is reached.

V. RESULTS AND DISCUSSIONS

In this section, the efficiency of the proposed BSCA on solving 0-1 knapsack problems is evaluated. The parameter settings of the proposed BSCA as well as the other comparative algorithms are provided in Sect. V-A, while the characteristics of the datasets are given in Sect V-B. The results of the proposed BSCA against the other comparative algorithms are reported in Sect. V-C.

The proposed BSCA algorithm was run on a Windows 10 Home 64-bit operating system, Intel Core i7 2.8 GHz processor mated with 16 GB of RAM. The algorithm was implemented on MATLAB as a programming language.

A. Parameter settings

The parameter settings of the proposed BSCA, as well as the GA and PSO are similar to BFPA introduced in [22] in order to make fair comparisons between the comparative algorithms. The parameter settings of these algorithms are given in Table I.

TABLE I
PARAMETER SETTINGS OF BSCA, GA, PSO, AND BFPA.

Algorithm	Parameter	Value
GA	Max iterations	1000
	Crossover Rate	0.5
	Mutation Rate	0.03
	Population Size	20
PSO	Max iterations	1000
	W	0.9
	C1 = C2	1
	Population Size	20
BFPA	Max iterations	1000
	P	0.8
	λ	1.5
	Population Size	20
BSCA	Max iterations	1000
	Population Size	20

B. Datasets characteristics

Table II illustrates the dimension, capacity, weight and profit vectors for the 0-1 knapsack instances to be optimized knowing that L1 to L10 were extracted from the low-dimensional instances found at [15], [16] and L11 to L15 from [17].

C. Comparison with others

Table III illustrates the results of the proposed BSCA against the GA, PSO, and BFPA algorithms. It should be noted that, the BSCA, GA, and PSO algorithms were run 30 times continuously whereas the results of the BFPA are taken from [22]. The experimental results are summarized in terms of the best, average and worst solution acquired for total runs, standard deviation (SD) of the acquired solutions over runs and success rate (SR) which depicts the number of optimal solutions acquired over the runs. Number of iterations to optimum solution and time to optimal solution (in seconds) was measured to further compare speed performance measures of the four algorithms.

As shown in Table III, BFPA has shown superior performance over any of the proposed optimization algorithms for all the performance metrics especially as the dimension of knapsack instances increased. It is worth mentioning that GA was relatively immune to the increasing dimension of knapsack instances and could still get better results compared to that of the implemented PSO and BSCA. However, BSCA was able to converge to an optimum faster than any of GA and PSO, and was in fact close to the that of BFPA or has outrun BFPA for some instances. However, one should keep in mind that BFPA was executed in JAVA whereas the other algorithms were coded on MATLAB which might have played a role in many factors depending on how the code is executed.

VI. CONCLUSION AND FUTURE DEVELOPMENTS

This paper analyzed SCA in its ability to solve different 0-1 knapsack problems. The proposed algorithm is called BSCA. The 0-1 knapsack problem is an constraint-based optimization problem in which a number of items have to be packed into a container by maximizing the value of the items in the container while also adhering to the weight limit of the container. When optimizing 0-1 knapsack optimization instances, BSCA illustrated superior performance over GA and PSO for most of the mentioned 0-1 knapsack instances and over BFPA for some but proved to be extremely close in terms of runtime to optimum for the rest of the instances. However, GA and BFPA showed capability of acquiring better results for knapsack problems with a relatively higher dimension compared to others. The knapsack problem has many different variations and accordingly, a future research could delve deeper into analyzing the performance of the proposed BSCA algorithm on different knapsack problems with different sizes and complexities. In addition, the performance of the proposed BSCA can be enhanced by hybridizing it with local-search algorithm like β -hill climbing [26]. Finally, we can investigate the efficiency of the proposed BSCA for other optimizations problems like nurse restoring [27], university timetabling [28], feature selection [29], etc.

REFERENCES

- [1] A. J. Kulkarni, G. Krishnasamy, and A. Abraham, *Cohort intelligence: a socio-inspired optimization method*. Springer, 2017.

TABLE II
THE CHARACTERISTICS OF THE 0-1 KNAPSACK DATASETS

Instance	Dimension	Capacity	Weights	Profit
L1	10	269	w = 95, 4, 60, 32, 23, 72, 80, 62, 65, 46	p = 55, 10, 47, 5, 4, 50, 8, 61, 85, 87
L2	20	878	w = 92, 4, 43, 83, 84, 68, 92, 82, 6, 44, 32, 18, 56, 83, 25, 96, 70, 48, 14, 58	p = 44, 46, 90, 72, 91, 40, 75, 35, 8, 54, 78, 40, 77, 15, 61, 17, 75, 29, 75, 63
L3	4	20	w = 6, 5, 9, 7	p = 9, 11, 13, 15
L4	4	11	w = 2, 4, 6, 7	p = 6, 10, 12, 13
L5	15	375	w = 56.358531, 80.874050, 47.987304, 89.596240, 74.660482, 85.894345, 51.353496, 1.498459, 36.445204, 16.589862, 44.569231, 0.466933, 37.788018, 57.118442, 60.716575	p = 0.125126, 19.330424, 58.500931, 35.029145, 82.284005, 17.410810, 71.050142, 30.399487, 9.140294, 14.731285, 98.852504, 11.908322, 0.891140, 53.166295, 60.176397
L6	10	60	w = 30, 25, 20, 18, 17, 11, 5, 2, 1, 1	p = 20, 18, 17, 15, 15, 10, 5, 3, 1, 1
L7	7	50	w = 31, 10, 20, 19, 4, 3, 6	p = 70, 20, 39, 37, 7, 5, 10
L8	23	10,000	w = 983, 982, 981, 980, 979, 978, 488, 976, 972, 486, 486, 972, 972, 485, 485, 969, 966, 483, 964, 963, 961, 958, 959	p = 981, 980, 979, 978, 977, 976, 487, 974, 970, 485, 485, 970, 970, 484, 484, 976, 974, 482, 962, 961, 959, 958, 857
L9	5	80	w = 15, 20, 17, 8, 31	p = 33, 24, 36, 37, 12
L10	20	879	w = 84, 83, 43, 4, 44, 6, 82, 92, 25, 83, 56, 18, 58, 14, 48, 70, 96, 32, 68, 92	p = 91, 72, 90, 46, 55, 8, 35, 75, 61, 15, 77, 40, 63, 75, 29, 75, 17, 78, 40, 44
L11	30	577	w = 46, 17, 35, 1, 26, 17, 17, 48, 38, 17, 32, 21, 29, 48, 31, 8, 42, 37, 6, 9, 15, 22, 27, 14, 42, 40, 14, 31, 6, 34	p = 57, 64, 50, 6, 52, 6, 85, 60, 70, 65, 63, 96, 18, 48, 85, 50, 77, 18, 70, 92, 17, 43, 5, 23, 67, 88, 35, 3, 91, 48
L12	35	655	w = 7, 4, 36, 47, 6, 33, 8, 35, 32, 3, 40, 50, 22, 18, 3, 12, 30, 31, 13, 33, 4, 48, 5, 17, 33, 26, 27, 19, 39, 15, 33, 47, 17, 41, 40	p = 35, 67, 30, 69, 40, 40, 21, 73, 82, 93, 52, 20, 61, 20, 42, 86, 43, 93, 38, 70, 59, 11, 42, 93, 6, 39, 25, 23, 36, 93, 51, 81, 36, 46, 96
L13	40	819	w = 28, 23, 35, 38, 20, 29, 11, 48, 26, 14, 12, 48, 35, 36, 33, 39, 30, 26, 44, 20, 13, 15, 46, 36, 43, 19, 32, 2, 47, 24, 26, 39, 17, 32, 17, 16, 33, 22, 6, 12	p = 13, 16, 42, 69, 66, 68, 1, 13, 77, 85, 75, 95, 92, 23, 51, 79, 53, 62, 56, 74, 7, 50, 23, 34, 56, 75, 42, 51, 13, 22, 30, 45, 25, 27, 90, 59, 94, 62, 26, 11
L14	45	907	w = 18, 12, 38, 12, 23, 13, 18, 46, 1, 7, 20, 43, 11, 47, 49, 19, 50, 7, 39, 29, 32, 25, 12, 8, 32, 41, 34, 24, 48, 30, 12, 35, 17, 38, 50, 14, 47, 35, 5, 13, 47, 24, 45, 39, 1	p = 98, 70, 66, 33, 2, 58, 4, 27, 20, 45, 77, 63, 32, 30, 8, 18, 73, 9, 92, 43, 8, 58, 84, 35, 78, 71, 60, 38, 40, 43, 43, 22, 50, 4, 57, 5, 88, 87, 34, 98, 96, 99, 16, 1, 25
L15	50	882	w = 15, 40, 22, 28, 50, 35, 49, 5, 45, 3, 7, 32, 19, 16, 40, 16, 31, 24, 15, 42, 29, 4, 14, 9, 29, 11, 25, 37, 48, 39, 5, 47, 49, 31, 48, 17, 46, 1, 25, 8, 16, 9, 30, 33, 18, 3, 3, 3, 4, 1	p = 78, 69, 87, 59, 63, 12, 22, 4, 45, 33, 29, 50, 19, 94, 95, 60, 1, 91, 69, 8, 100, 32, 81, 47, 59, 48, 56, 18, 59, 16, 45, 54, 47, 84, 100, 98, 75, 20, 4, 19, 58, 63, 37, 64, 90, 26, 29, 13, 53, 83

- [2] D. E. Goldberg and J. H. Holland, "Genetic algorithms and machine learning," 1988.
- [3] R. Eberhart and J. Kennedy, "A new optimizer using particle swarm theory," in *MHS'95. Proceedings of the Sixth International Symposium on Micro Machine and Human Science*. Ieee, 1995, pp. 39–43.
- [4] F. MiarNaeimi, G. Azizyan, and M. Rashki, "Horse herd optimization algorithm: A nature-inspired algorithm for high-dimensional optimization problems," *Knowledge-Based Systems*, vol. 213, p. 106711, 2021.
- [5] S. He, Q. H. Wu, and J. Saunders, "Group search optimizer: an optimization algorithm inspired by animal searching behavior," *IEEE transactions on evolutionary computation*, vol. 13, no. 5, pp. 973–990, 2009.
- [6] S. Mirjalili, A. H. Gandomi, S. Z. Mirjalili, S. Saremi, H. Faris, and S. M. Mirjalili, "Salp swarm algorithm: A bio-inspired optimizer for engineering design problems," *Advances in Engineering Software*, vol. 114, pp. 163–191, 2017.
- [7] R. A. Zitar, M. A. Al-Betar, M. A. Awadallah, I. A. Doush, and K. As-saleh, "An intensive and comprehensive overview of jaya algorithm, its versions and applications," *Archives of Computational Methods in Engineering*, pp. 1–30, 2021.
- [8] M. A. Al-Betar, Z. A. A. Alyasseri, M. A. Awadallah, and I. A. Doush, "Coronavirus herd immunity optimizer (chio)," *Neural Computing and Applications*, vol. 33, no. 10, pp. 5011–5042, 2021.
- [9] S. Mirjalili, "Sca: a sine cosine algorithm for solving optimization problems," *Knowledge-based systems*, vol. 96, pp. 120–133, 2016.
- [10] L. Abualigah and A. Diabat, "Advances in sine cosine algorithm: a comprehensive survey," *Artificial Intelligence Review*, vol. 54, pp. 2567–2608, 2021.
- [11] E. Yassien, R. Masadeh, A. Alzaqebah, and A. Shaheen, "Grey wolf optimization applied to the 0/1 knapsack problem," *International Journal of Computer Applications*, vol. 169, no. 5, pp. 11–15, 2017.
- [12] H. S. Alamri, K. Z. Zamli, M. F. A. Razak, and A. Firdaus, "Solving 0/1 knapsack problem using opposition-based whale optimization algorithm (owoa)," in *Proceedings of the 2019 8th International Conference on Software and Computer Applications*, 2019, pp. 135–139.
- [13] T. K. Truong, K. Li, and Y. Xu, "Chemical reaction optimization with greedy strategy for the 0–1 knapsack problem," *Applied soft computing*, vol. 13, no. 4, pp. 1774–1780, 2013.
- [14] Y. Feng, G.-G. Wang, S. Deb, M. Lu, and X.-J. Zhao, "Solving 0–1 knapsack problem by a novel binary monarch butterfly optimization," *Neural computing and applications*, vol. 28, no. 7, pp. 1619–1634, 2017.
- [15] D. Kreher and D. Simpson, *Combinatorial Algorithms*. CRC Press, 1998.
- [16] S. Martello, "Knapsack problems: algorithms and computer implementations," *Wiley-Interscience series in discrete mathematics and optimization*, 1990.
- [17] A. J. Kulkarni, G. Krishnasamy, and A. Abraham, "Solution to 0–1 knapsack problem using cohort intelligence algorithm," in *Cohort Intelligence: A Socio-inspired Optimization Method*. Springer, 2017, pp. 55–74.
- [18] R. P. Singh, "Solving 0–1 knapsack problem using genetic algorithms," in *2011 IEEE 3rd International Conference on Communication Software and Networks*. IEEE, 2011, pp. 591–595.
- [19] T. Y. Lim, M. A. Al-Betar, and A. T. Khader, "Taming the 0/1 knapsack problem with monogamous pairs genetic algorithm," *Expert Systems with Applications*, vol. 54, pp. 241–250, 2016.
- [20] Z. Li and N. Li, "A novel multi-mutation binary particle swarm optimization for 0/1 knapsack problem," in *2009 Chinese Control and Decision Conference*. IEEE, 2009, pp. 3042–3047.
- [21] M. Abdel-Basset, Q. Luo, F. Miao, and Y. Zhou, "Solving 0–1 knapsack problems by binary dragonfly algorithm," in *International conference on intelligent computing*. Springer, 2017, pp. 491–502.
- [22] M. Abdel-Basset, D. El-Shahat, and I. El-Henawy, "Solving 0–1 knap-

- sack problem by binary flower pollination algorithm,” *Neural Computing and Applications*, vol. 31, no. 9, pp. 5477–5495, 2019.
- [23] J. Cao, B. Yin, X. Lu, Y. Kang, and X. Chen, “A modified artificial bee colony approach for the 0-1 knapsack problem,” *Applied Intelligence*, vol. 48, no. 6, pp. 1582–1595, 2018.
 - [24] M. Abdel-Basset, R. Mohamed, R. K. Chakraborty, M. Ryan, and S. Mirjalili, “New binary marine predators optimization algorithms for 0–1 knapsack problems,” *Computers & Industrial Engineering*, vol. 151, p. 106949, 2021.
 - [25] R. Hinterding, “Representation, constraint satisfaction and the knapsack problem,” in *Proceedings of the 1999 Congress on Evolutionary Computation-CEC99 (Cat. No. 99TH8406)*, vol. 2. IEEE, 1999, pp. 1286–1292.
 - [26] M. A. Al-Betar, “ β -hill climbing: an exploratory local search,” *Neural Computing and Applications*, vol. 28, no. 1, pp. 153–168, 2017.
 - [27] M. A. Awadallah, A. L. Bolaji, and M. A. Al-Betar, “A hybrid artificial bee colony for a nurse rostering problem,” *Applied Soft Computing*, vol. 35, pp. 726–739, 2015.
 - [28] M. A. Al-Betar, A. T. Khader, and M. Zaman, “University course timetabling using a hybrid harmony search metaheuristic algorithm,” *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 42, no. 5, pp. 664–681, 2012.
 - [29] M. A. Awadallah, M. A. Al-Betar, A. I. Hammouri, and O. A. Alomari, “Binary jaya algorithm with adaptive mutation for feature selection,” *Arabian Journal for Science and Engineering*, vol. 45, no. 12, pp. 10 875–10 890, 2020.

TABLE III
THE COMPARISON RESULTS.

Instance	Optimal	Algorithm	Best	Average	Worst	SD	SR	No. of Iterations	Time (s)
L1	295	GA	295	294.9667	294	0.1826	0.9667	16	0.08915
	295	PSO	295	295	295	0	1	32	0.0588
	295	BFPA	295	295	295	0	1	20	0.0216
	295	BSCA	295	295	295	0	1	2	0.006205
L2	1024	GA	1024	1024	1024	0	1	127	0.746382
	1024	PSO	1024	1000.2	943	25.4541	0.1667	197	0.2269
	1024	BFPA	1024	1024	1024	0	1	500	0.2659
	1024	BSCA	1024	978.5667	922	23.1631	0.0333	755	0.276971
L3	35	GA	35	35	35	0	1	1	0.001068
	35	PSO	35	35	35	0	1	1	0.004948
	35	BFPA	35	35	35	0	1	1	0.0035
	35	BSCA	35	35	35	0	1	2	0.005023
L4	23	GA	23	23	23	0	1	1	0.001857
	23	PSO	23	23	23	0	1	1	0.004628
	23	BFPA	23	23	23	0	1	1	0.0032
	23	BSCA	23	23	23	0	1	1	0.001776
L5	481.0694	GA	481.0694	475.7347	321.0303	29.219	0.9667	45	0.21315
	481.0694	PSO	481.0694	450.9676	429.2825	15.9734	0.1	216	0.246178
	481.0694	BFPA	481.0694	481.0694	481.0694	0	1	50	0.0349
	481.0694	BSCA	481.0694	471.7306	430.9244	15.0634	0.5	595	0.269844
L6	52	GA	52	51.7	51	0.4661	0.7	7	0.03301
	52	PSO	52	51.4667	51	0.5074	0.4667	5	0.027398
	52	BFPA	52	52	52	0	1	50	0.0151
	52	BSCA	52	52	52	0	1	1	0.002365
L7	107	GA	107	107	107	0	1	75	0.368315
	107	PSO	107	105.3	102	2.0367	0.5	5	0.006018
	107	BFPA	107	107	107	0	1	2	0.0059
	107	BSCA	107	107	107	0	1	5	0.008811
L8	9767	GA	9767	9766.8	9765	0.5509	0.8667	250	1.198792
	9767	PSO	9767	9759.4	9735	7.9076	0.1333	247	0.323553
	9767	BFPA	9767	9767	9767	0	1	500	0.4187
	9767	BSCA	9767	9759.3	9750	3.9246	0.0333	184	0.068833
L9	130	GA	130	130	130	0	1	1	0.000832
	130	PSO	130	129.6	118	2.1909	0.9667	1	0.005206
	130	BFPA	130	130	130	0	1	2	0.0047
	130	BSCA	130	130	130	0	1	7	0.011261
L10	1025	GA	1025	1024.8	1019	1.0954	0.9667	36	0.177431
	1025	PSO	1025	1005	965	13.0687	0.0333	143	0.177193
	1025	BFPA	1025	1025	1025	0	1	500	0.1725
	1025	BSCA	1025	984.9333	942	19.5835	0.0667	454	0.174825
L11	1437	GA	1437	1428.4	1407	9.224	0.4667	474	2.272847
	1437	PSO	1420	1369.7	1297	28.41	0	N/A	N/A
	1437	BFPA	1437	1437	1437	0	1	20	0.1049
	1437	BSCA	1380	1322.6	1282	21.899	0	N/A	N/A
L12	1689	GA	1689	1680.3	1661	8.023	0.2333	522	2.467878
	1689	PSO	1663	1563.8	1458	47.0337	0	N/A	N/A
	1689	BFPA	1689	1689	1689	0	1	20	0.0662
	1689	BSCA	1621	1500.9	1444	38.113	0	N/A	N/A
L13	1821	GA	1821	1807.8	1781	9.8355	0.1	773	3.732068
	1821	PSO	1734	1677.6	1581	43.4175	0	N/A	N/A
	1821	BFPA	1821	1821	1821	0	1	100	0.2981
	1821	BSCA	1753	1632.4	1541	56.9553	0	N/A	N/A
L14	2033	GA	2030	2009.8	1976	15.6456	0	N/A	N/A
	2033	PSO	1946	1828.9	1712	56.9894	0	N/A	N/A
	2033	BFPA	2033	2033	2033	0	1	50	0.1676
	2033	BSCA	1858	1749.7	1661	44.1917	0	N/A	N/A
L15	2440	GA	2438	2403.4	2362	24.0037	0	N/A	N/A
	2440	PSO	2256	2123.6	2002	64.4316	0	N/A	N/A
	2440	BFPA	2440	2440	2440	0	1	100	0.2303
	2440	BSCA	2171	2037.3	1973	51.2245	0	N/A	N/A